

Phase 5: Event Handling with Superdense Time

Master
Algorithm

Event Source

Combi
(source + listener)

Listener

Input from Phase 4:
- All components at t_event (real time)
- initial_events = [(comp_name, event_name), ...]
- Example: [("Source", "zero_crossing")]

Goal:
Handle events iteratively at superdense time
(t_event, 0), (t_event, 1), ... until no new events

Initialization

Initialize superdense time:

current_time = DenseTime(t=t_event, micro=0)
event_pairs = initial_events
all_handled_events = set()

Superdense Time Loop

loop [While event_pairs exist AND current_time.micro < max_microsteps]

Record Events in History

loop [For each event pair]

system.history.record_event(
comp_name, event_name, current_time)

Example:
("Source", "zero_crossing", DenseTime(t=0.333, micro=0))

Group Events by Listener

Build event dispatch map:

Example from Case 2:
events_by_component = {"Combi": ["zero_crossing"], "Listener": []}

Check Event Commutativity (if multiple events per listener)

Only for len(event_names) > 1

alt [Method 1: Annotation-based]

Check comp.event_commutativity
{{'v_invert', 'v_double': True}}

alt [All pairs commute]

Verified via annotations

[Non-commutative]

ERROR: Cannot handle simultaneously

[Method 2: Dynamic verification]

snapshot # test all permutations # compare states

alt [Order-independent result]

Commutative (tested)

[Order affects result]

ERROR: Non-commutative

Evaluate Indicators Before Handling

loop [For each event source]

evaluate_event_indicators()
indicators_before[Src.name]

Dispatch Events to Listeners

loop [For each (comp_name, event_name) in event_pairs]

targets = system_event_targets_by_source(
comp_name, event_name)

loop [For each target_name in targets]

handle_event((event_name), t_event)

handle_events_internal((event_name), t_event)

update_output_states(t_event, [event_name])

record_outputs(t_event)

event handled

Prepare Inputs After Handling

_prepare_inputs(system, t_event)

Ensures consistency before detecting new events

Evaluate Indicators After Handling

loop [For each event source]

evaluate_event_indicators()
indicators_after[Combi.name]

Detect New Events Triggered by Handlers

new_events = []

loop [For each event source component]

Detect crossings:
events = comp.detect_event_crossing(
indicators_before[comp.name],
indicators_after[comp.name])

loop [For each detected event_name]

event_pair = (comp.name, event_name)

alt [event_pair not in all_handled_events]

new_events.append(event_pair)

Example from Case 2:
new_events = [("Combi", "v_change")] # handled at (t_event, 1)

Advance Microstep or Exit

alt [new_events exist]

Event chain detected!

event_pairs = new_events
current_time = (t_event, micro+1)

[No new events]

Exit microstep loop

Termination Check

alt [current_time.micro >= max_microsteps]

ERROR: Zeno Behavior

Raise RuntimeError

Finalization

t_left # t_event

Return to Master

Master
Algorithm

Event Source

Combi
(source + listener)

Listener